

Step 9 — Correlation Heatmap

Why?

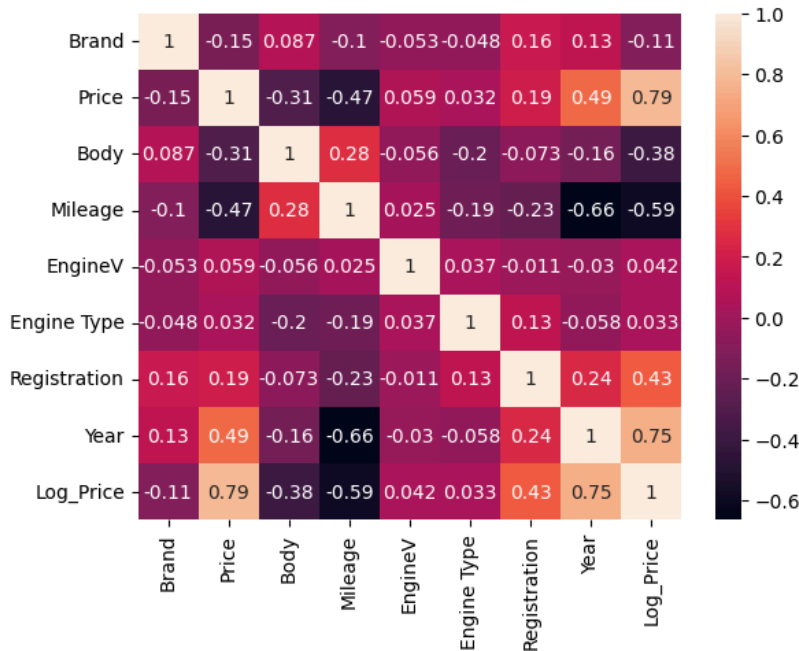
A heatmap shows how strongly each feature is related to `log_price`.

- Value close to **+1** → strong positive relationship (feature goes up, price goes up)
- Value close to **-1** → strong negative relationship (feature goes up, price goes down)
- Value close to **0** → weak/no relationship

💡 This helps us understand which features are most important for prediction.

```
sns.heatmap(df.corr(), annot=True)
```

<Axes: >



Step 10 — Split Data into Train and Test Sets

Why?

We train the model on one part of the data, and test it on a *different* part it has never seen.

This tells us how well the model performs on **new, unseen data** — which is the real goal!

Split	Purpose
80% Train	Model learns from this
20% Test	We evaluate model here

💡 `random_state=42` ensures same split every time you run the code.

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=['Log_Price'])
y = df['Log_Price']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
X_train.head()
```

	Brand	Price	Body	Mileage	EngineV	Engine Type	Registration	Year	
2882	6	5300.0	4	259	1.6	1	1	1995	
4035	2	5000.0	3	240	2.2	0	0	2004	
2313	6	34000.0	0	92	3.0	0	1	2012	
1533	0	62000.0	4	24	3.0	0	1	2015	
305	5	13900.0	1	90	1.6	3	1	2012	

Next steps: [Generate code with X_train](#) [New interactive sheet](#)

```
X_train.shape
```

```
(3220, 8)
```

```
X_test.shape
```

```
(805, 8)
```

```
y_train.shape
```

```
(3220,)
```

```
y_test.shape
```

```
(805,)
```

▼ Step 11 — Linear Regression

Why start here?

Linear Regression is the simplest and most foundational regression model.

It finds the best straight line (or plane in multiple dimensions) that fits the data.

Formula:

`Price = b0 + b1×Year + b2×Mileage + b3×EngineV + ...`

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

LinearRegression()

```
model.score(X_train, y_train)
```

```
0.8795283878546256
```

```
y_predict = model.predict(X_test)
```

```
y_predict[:10]
```

```
array([ 9.62616148,  9.34794832,  9.49242443,  9.57207904,  8.74212921,  
       9.30507308, 11.71253058, 12.56727337, 10.19644372,  7.39162295])
```

```
compare = pd.DataFrame({  
    'Actual': y_test,  
    'Predicted': y_predict  
})  
compare.head(10)
```

	Actual	Predicted	
1862	10.043249	9.626161	
1295	9.093807	9.347948	
2946	9.568015	9.492424	
2093	9.024011	9.572079	
1318	9.047821	8.742129	
502	9.230143	9.305073	
3858	11.707670	11.712531	
594	12.010666	12.567273	
903	10.425253	10.196444	
4319	8.612503	7.391623	

Next steps: [Generate code with compare](#) [New interactive sheet](#)

✓ Step 12 — Ridge Regression

Why Ridge?

Ridge is Linear Regression with a **penalty** added.

When we have many features, some coefficients can get very large — this causes **overfitting** (model memorizes training data but fails on new data).

Ridge **shrinks** large coefficients without removing any feature completely.

```
from sklearn.linear_model import Ridge  
  
ridge_model = Ridge()  
ridge_model.fit(X_train, y_train)
```

▼ Ridge ⓘ ?
Ridge()

```
ridge_model.score(X_train, y_train)
```

```
0.8795275304934556
```

```
ridge_y_predict = ridge_model.predict(X_test)
```

```
compare = pd.DataFrame({  
    'Actual': y_test,  
    'Predicted': ridge_y_predict  
})  
compare.head(10)
```

	Actual	Predicted
1862	10.043249	9.626285
1295	9.093807	9.347802
2946	9.568015	9.492009
2093	9.024011	9.571869
1318	9.047821	8.741831
502	9.230143	9.304798
3858	11.707670	11.712720
594	12.010666	12.567877
903	10.425253	10.196220
4319	8.612503	7.390702



Next steps:

[Generate code with compare](#)[New interactive sheet](#)

```
r2_score(y_test, ridge_y_predict)
```

```
0.8753267908774224
```